Lecture Notes: Polynomial Features

CIC1205 - Machine Learning

Prof. Eduardo Bezerra - Cefet/RJ

April 12, 2026

0 Contents

1 Introduction and Motivation **2**
1.1 Recap: Linear and Logistic Regression . . . 2
1.2 The Limitation of Linear Models . . . 2
1.3 Two Paths Forward . . . 3

2 Feature Interactions and Polynomial Features **3**
2.1 The Core Idea . . . 4
2.2 Polynomial Feature Expansion . . . 4
2.3 A Concrete Example: Three Features, Degree 3 . . . 5

3 Polynomial Features for Regression **6**
3.1 From Linear to Polynomial Regression . . . 6
3.2 Linear in the Parameters: A Critical Distinction . . . 6
3.3 The Design Matrix with Polynomial Features . . . 7
3.4 Geometric Intuition . . . 7
3.5 Worked Example: Degree-2 Polynomial Regression . . . 8

4 Polynomial Features for Classification **9**
4.1 Non-Linear Decision Boundaries . . . 9
4.2 Circular Decision Boundary: An Illustrative Example . . . 9
4.3 More Complex Boundaries . . . 10

5 Overfitting and the Bias–Variance Trade-Off **12**
5.1 The Two Failure Modes . . . 12
5.2 The Bias–Variance Trade-Off . . . 12
5.3 Model Selection via Validation Error . . . 12

6 Regularisation **14**
6.1 Motivation . . . 14
6.2 L2 Regularisation (Ridge) . . . 15
6.3 Regularised Gradient Descent . . . 15

7 Feature Scaling **16**
7.1 Why Feature Scaling Matters More with Polynomial Features . . . 16
7.2 Practical Guidelines for Scaling with Polynomial Features . . . 16

8 The ML Pipeline with Polynomial Features **17**
8.1 Where This Fits in the Learning Workflow . . . 17

9 Feature Engineering Beyond Polynomials **18**
9.1 The Broader Picture . . . 18
9.2 A Forward Look: Kernel Methods . . . 18
9.3 Limitations and When to Move On . . . 18

10 Summary **19**

1 Introduction and Motivation

1.1 Recap: Linear and Logistic Regression

In previous lectures of notes, we studied two foundational supervised learning algorithms: linear regression and logistic regression.

In linear regression, the hypothesis was chosen to be

$$h_\theta(x) = \boldsymbol{\theta}^T \mathbf{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n,$$

producing a real-valued output $h_\theta(x) \in \mathbb{R}$. The cost function was the Mean Squared Error (MSE), and the parameter vector $\boldsymbol{\theta}$ was found by minimising this cost using gradient descent.

In logistic regression, the hypothesis was

$$h_\theta(x) = \sigma\left(\boldsymbol{\theta}^T \mathbf{x}\right) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}},$$

which outputs a probability in $(0, 1)$ suitable for binary classification. The cost function was the log-loss (binary cross-entropy), and the parameters were again found via gradient descent.

Despite their differences, both algorithms share an important structural property: in their *vanilla* forms, they produce *linear decision boundaries*. For regression, this means the fitted model is a straight line (or a hyperplane in higher dimensions). For classification, this means the boundary separating the two classes is a straight line (or hyperplane).

1.2 The Limitation of Linear Models

Linear models are fast, interpretable, and often surprisingly effective. However, many real-world datasets exhibit non-linear structure that a straight line simply cannot capture.

Intuition

A linear model can only *tilt* a line or a hyperplane; it cannot *bend* it. If the true relationship is curved (for instance, U-shaped), no combination of parameter values $\theta_0, \theta_1, \ldots$ will fix the mismatch. The problem is not how the parameters are tuned; the problem is that the model class itself is too restricted to represent the truth.

Consider a regression dataset where the target variable y follows a curved trend as a function of x. A linear hypothesis $h_\theta(x) = \theta_0 + \theta_1 x$ would underfit: it cannot represent curvature, leading to systematically large errors for all parameter choices.

In binary classification, it frequently occurs that the two classes are not *linearly separable*: no single straight line can perfectly separate them. Applying vanilla logistic regression in such a case leads to poor classification performance regardless of how well the parameters are optimised.

Figure 1 reinforces the key limitation of linear models. Changing the parameters of a linear hypothesis can only translate or rotate a line; it cannot produce curvature. This is why a genuinely curved relationship cannot be captured well by vanilla linear regression, no matter how carefully the parameters are optimised.

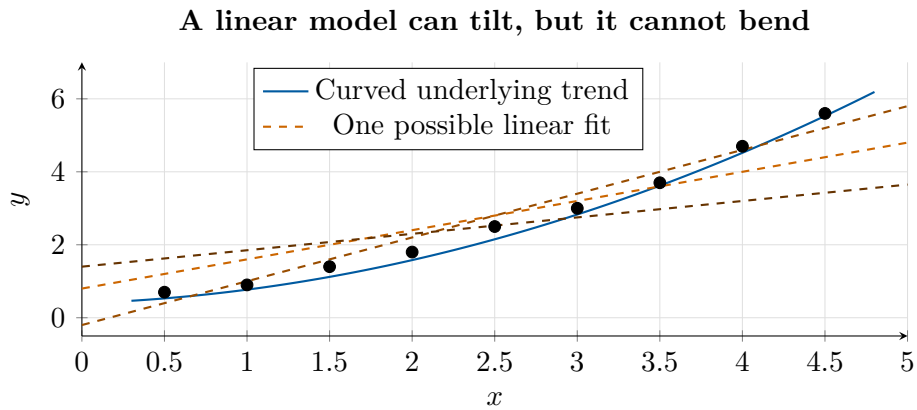

Figure 1: Illustration of the expressive limitation of a linear model. By changing θ_0 and θ_1, the fitted line can move up, down, or rotate, but it remains a straight line. If the true relationship is curved, no choice of linear parameters can make the model bend to match that structure.

Linear Separability

A binary classification dataset is **linearly separable** if there exists a hyperplane that perfectly separates the examples of class 0 from the examples of class 1. If no such hyperplane exists, the dataset is **not linearly separable**, and a linear classifier will inevitably misclassify some training examples.

1.3 Two Paths Forward

When linear models are insufficient, there are broadly two strategies:

1. **Use a more complex learning algorithm.** Decision trees, support vector machines, and neural networks can all learn non-linear patterns directly. We will study these in later notes.
2. **Engineer new features.** Keep the linear/logistic regression framework, but enrich the input representation by constructing new features that capture non-linear relationships. This approach is the subject of the present notes.

The second strategy has a deceptively simple but powerful mathematical foundation.

Key Idea

We do not change the learning algorithm. We change the representation of the input. Instead of learning on x, we learn on $(x,\ x^2,\ x^3,\ \ldots)$. The gradient descent update rules, the cost function, and the optimisation procedure remain exactly the same. Only what we feed into them changes.

This idea (transforming inputs so that a simple model can express complex relationships) is one of the most important conceptual tools in machine learning.

2 Feature Interactions and Polynomial Features

2.1 The Core Idea

The fundamental idea is straightforward: given an original set of features $x_1, x_2, \ldots, x_n$, we *transform* them into an enriched set that includes products and powers of the original features. In simple terms, we include every possible product of features whose total degree does not exceed a chosen maximum d. The learning algorithm then operates on this larger feature space, where linear relationships can capture non-linear patterns in the original space.

Feature Interaction

A **feature interaction** is a new feature formed by multiplying two or more original features together. The resulting feature $x_i^p \cdot x_j^q$ informs the learning algorithm about the *joint* effect of x_i and x_j on the target, enabling the model to capture relationships that neither variable could capture individually.

Why interactions matter

Without an interaction feature $x_i x_j$, the model implicitly assumes:

"The effect of x_i on the target is the same regardless of the value of x_j."

With $x_i x_j$ included, the model can represent:

"The effect of x_i changes when x_j changes."

For example, the area of a rectangular room is the product of its length and its width. If a model predicting housing prices has length (x_1) and width (x_2) as separate features, it cannot compute area unless the product $x_1 x_2$ is included explicitly. No amount of learning on x_1 and x_2 alone will recover the multiplicative relationship.

Figure 2 highlights the conceptual role of an interaction feature. In a purely additive model, each input contributes independently to the prediction. Once a product term such as $x_1 x_2$ is added, the model can express dependence between inputs, allowing the effect of one variable to vary with the value of the other.

2.2 Polynomial Feature Expansion

Given n original features and a maximum polynomial degree d, the **polynomial feature expansion** produces all monomials of the form

$$x_1^{e_1} \cdot x_2^{e_2} \cdots x_n^{e_n}, \qquad e_i \geq 0, \quad e_1 + e_2 + \cdots + e_n \leq d.$$

Note

Since multiplication is commutative, $x_1 x_2$ and $x_2 x_1$ are the same feature. Each distinct monomial appears exactly once in the expanded feature vector; there is no redundancy.

Polynomial Feature Expansion

Starting from n features $x_1, \ldots, x_n$ and a degree d, the expanded feature vector includes:

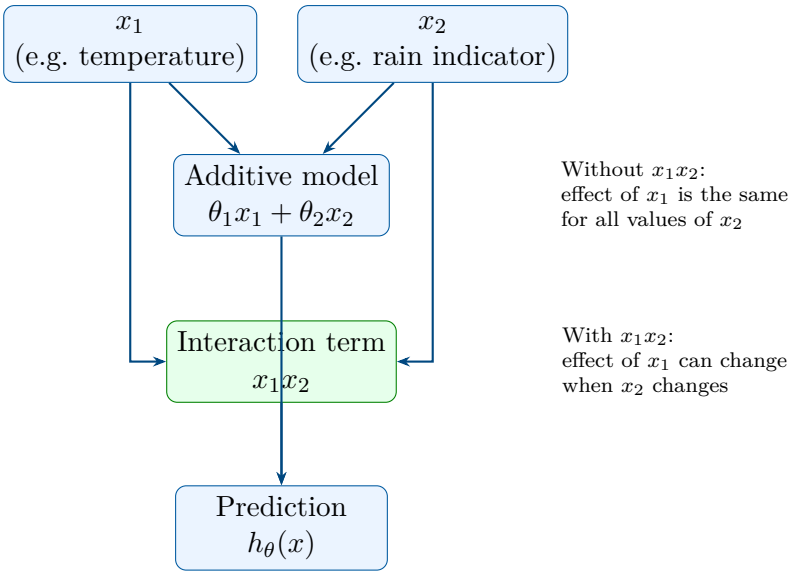

Figure 2: Schematic view of a feature interaction. In an additive model, the contribution of x_1 does not depend on the value of x_2. Once the product $x_1 x_2$ is added, the model can represent situations in which the effect of one feature changes according to the other.

- the bias term: $1 = x_1^0 = x_2^0 = \cdots = x_n^0$,
- the original features: $x_1, x_2, \ldots, x_n$,
- squared terms: $x_1^2, x_2^2, \ldots, x_n^2$,
- pairwise products: $x_1 x_2,\ x_1 x_3,\ \ldots,\ x_{n-1} x_n$,
- higher-order terms: $x_1^3,\ x_1^2 x_2,\ x_1 x_2 x_3,\ \ldots$ up to degree d.

Mental Model

Polynomial features do not make the learning algorithm smarter.
They make the input richer, so that a simple linear model can express more complex relationships. The algorithm does exactly the same thing as before: it just operates on a more informative representation of the data.

2.3 A Concrete Example: Three Features, Degree 3

To make the expansion concrete, suppose we have three original features x_1, x_2, x_3 and set the maximum polynomial degree to $d = 3$. The generated features include:

Degree	**Features**
0 (constant)	1
1 (original)	$x_1,\ x_2,\ x_3$
2 (squared and pairwise)	$x_1^2,\ x_2^2,\ x_3^2,\ x_1 x_2,\ x_1 x_3,\ x_2 x_3$
3 (cubic and mixed)	$x_1^3,\ x_2^3,\ x_3^3,\ x_1^2 x_2,\ x_1^2 x_3,\ x_2^2 x_1,$ $x_2^2 x_3,\ x_3^2 x_1,\ x_3^2 x_2,\ x_1 x_2 x_3$

Starting from just 3 original features, we obtain 20 features in total (including the constant term). In general, the number of features grows rapidly with both n and d.

Combinatorial Growth

The number of monomials of degree at most d in n variables is $\binom{n+d}{d}$. For $n = 3, d = 3$: $\binom{6}{3} = 20$. For $n = 10, d = 3$: $\binom{13}{3} = 286$. For $n = 10, d = 5$: $\binom{15}{5} = 3{,}003$. This rapid growth has important implications for computational cost and the risk of overfitting, which we address in Section 5.

Entry Ticket 1

Refer to the polynomial feature expansion.

1. Starting from a single feature x_1 and degree $d = 4$, list all the features in the expanded set. How many features are there in total (including the bias term)?
2. Starting from two features x_1 and x_2 and degree $d = 2$, list all features in the expanded set. Verify your count using the formula $\binom{n+d}{d}$.
3. A classmate argues: "The effect of temperature on sales is always the same, regardless of whether it is raining or not." What kind of feature would you add to the model to let the data decide whether this assumption is correct? Explain your reasoning.
4. **(Predict the behaviour)** Suppose we keep the polynomial degree d fixed at 2 and progressively add more original features $x_1, x_2, \ldots, x_n$. How does the total number of expanded features grow? Does this growth become a concern at some point?

3 Polynomial Features for Regression

3.1 From Linear to Polynomial Regression

In univariate linear regression, the hypothesis is $h_\theta(x) = \theta_0 + \theta_1 x$. This is a degree-1 polynomial in x. To fit curved data, we instead use a degree-d polynomial:

Polynomial Regression Hypothesis

$$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_d x^d \tag{1}$$

3.2 Linear in the Parameters: A Critical Distinction

The key observation is that Equation (1) is still **linear in the parameters** $\theta_0, \theta_1, \ldots, \theta_d$, even though it is non-linear in x.

Key Concept: What "Linear Model" Really Means

Linear model does *not* mean linear in x.
It means linear in the **parameters** θ.

To see this, define new features $\phi_1 = x$, $\phi_2 = x^2$, ..., $\phi_d = x^d$. Then the polynomial hypothesis becomes:

$$h_\theta(\phi) = \theta_0 + \theta_1\phi_1 + \theta_2\phi_2 + \cdots + \theta_d\phi_d,$$

which is precisely the standard linear regression hypothesis applied to the transformed features $(\phi_1, \phi_2, \ldots, \phi_d)$.
Even x^5, $x_1^2x_2^3$, or $\sin(x)$ are acceptable features, as long as they appear multiplied by a parameter, never raised to the power of one. The moment you write $h(x) = x^\theta$, linearity in the parameters is lost.

This observation has a very practical consequence: we can reuse *the exact same gradient descent algorithm* as in the linear regression notes. We simply pre-process the data by adding the polynomial columns to the design matrix before running the algorithm.

3.3 The Design Matrix with Polynomial Features

Each training example $x^{(i)}$ is expanded into a row of the design matrix. For a degree-2 polynomial ($d = 2$) with a single original feature, every example is now represented as:

$$\left(1,\ x^{(i)},\ (x^{(i)})^2\right),$$

so the design matrix $\mathbf{\Phi}$ has shape $m \times 3$ (one row per training example, three columns for the bias, x, and x^2). The gradient descent update rules from the linear regression notes apply directly to $\mathbf{\Phi}$, treating the three columns as if they were three independent features.

Figure 3 shows explicitly what polynomial pre-processing does to each training example. A raw input value is mapped to a richer feature representation, which then becomes a row of the design matrix. This makes clear why polynomial regression is still ordinary linear regression: the optimisation algorithm is unchanged, but it is applied to transformed inputs.

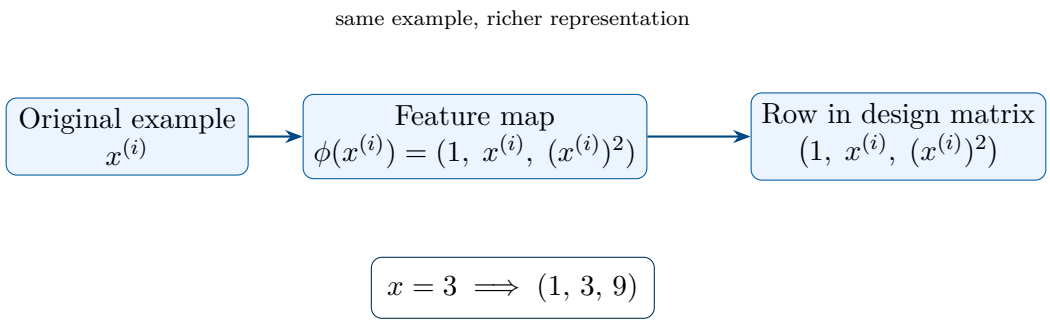

Figure 3: One training example being transformed into polynomial features before entering the design matrix. The learning algorithm does not operate directly on the raw scalar input x, but on its expanded representation $\phi(x)$.

3.4 Geometric Intuition

Figure 4 illustrates the difference between linear and polynomial regression. The training points exhibit a clear nonlinear trend, which cannot be adequately captured by a linear

model ($d = 1$), resulting in systematic underfitting. In contrast, the degree-2 polynomial model ($d = 2$) matches the curvature of the data and provides a substantially better fit. This example reinforces the idea that choosing an appropriate hypothesis class—here enabled by including the feature x^2—is essential for capturing the underlying structure of the data.

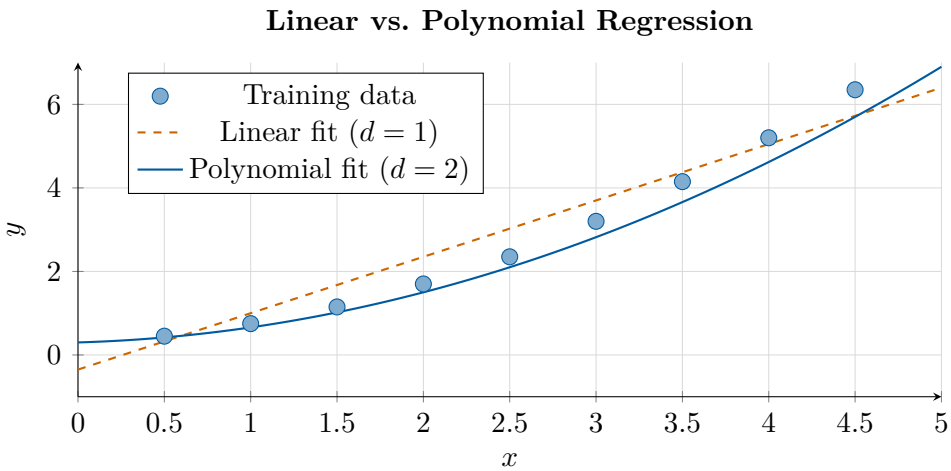

Figure 4: Comparison of a linear hypothesis (dashed orange) and a degree-2 polynomial hypothesis (solid blue) on a curved dataset. The linear model underfits, as it cannot capture the nonlinear trend and systematically deviates from the data. In contrast, the polynomial model matches the curvature and provides a much better fit by operating on the enriched feature space $(1, x, x^2)$.

3.5 Worked Example: Degree-2 Polynomial Regression

Suppose we have a single feature x and observe that the data appears to follow a quadratic trend. We expand the feature space to include x^2 and fit a degree-2 polynomial.

The training set, together with the expanded feature columns, is shown below.

i	$x^{(i)}$	$\phi_1^{(i)} = x^{(i)}$	$\phi_2^{(i)} = (x^{(i)})^2$
1	1.0	1.0	1.0
2	2.0	2.0	4.0
3	3.0	3.0	9.0
4	4.0	4.0	16.0
5	5.0	5.0	25.0

The hypothesis is $h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2$. We apply standard linear regression (gradient descent) to the three-column design matrix $[1,\ \phi_1,\ \phi_2]$, finding $\boldsymbol{\theta} = (\theta_0, \theta_1, \theta_2)$ that minimises the MSE cost function, which is identical to the cost from the linear regression notes, simply applied to the expanded features.

Entry Ticket 2

Refer to polynomial regression.

1. Suppose we use a degree-d polynomial for univariate regression. How many parameters θ_j does the model have? What is the shape of the design matrix for a training set of m examples?
2. Is the hypothesis $h_\theta(x) = \theta_0 + \theta_1 x^2 + \theta_2 x^4$ a linear model? Justify your answer carefully.
3. Is the hypothesis $h_\theta(x) = \theta_0 + x^{\theta_1}$ a linear model? How does it differ from the previous case?
4. **(Predict the behaviour)** We fit a degree-2 polynomial on a dataset that was generated by a degree-2 function plus random Gaussian noise. What do you expect to happen to the training error and the validation error as the dataset size m grows very large? Explain your reasoning.

4 Polynomial Features for Classification

4.1 Non-Linear Decision Boundaries

Recall from the logistic regression notes that the decision boundary is the set of points where $\boldsymbol{\theta}^T \mathbf{x} = 0$. With the original features, this is always a *hyperplane*, i.e., a straight line in 2D, a plane in 3D, and so on.

By introducing polynomial features, the decision boundary can become a curved surface in the original feature space, even though the model remains *linear* in the expanded space.

Geometric Insight

The decision boundary is curved in the original space (x_1, x_2), but it is a flat hyperplane in the expanded space $(x_1,\ x_2,\ x_1^2,\ x_2^2,\ \ldots)$.
Logistic regression always finds a hyperplane; polynomial features simply change the space in which that hyperplane lives.

Figure 5 summarises one of the most important geometric ideas in these notes. Polynomial logistic regression does not cease to be a linear classifier; rather, it becomes linear in a different feature space. What appears as a curved boundary in the original coordinates corresponds to a flat hyperplane after the polynomial transformation is applied.

4.2 Circular Decision Boundary: An Illustrative Example

Consider a two-dimensional classification problem with features x_1 and x_2. Suppose the positive class ($y = 1$) surrounds a cluster of the negative class ($y = 0$) at the origin. No straight line can separate these two classes.

Now add the polynomial features x_1^2 and x_2^2. The logistic regression hypothesis becomes:

$$h_\theta(x) = \sigma\Big(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2\Big)\,.$$

Suppose the learned parameters are $\boldsymbol{\theta}^T = (-1,\ 0,\ 0,\ 1,\ 1)$. The decision boundary is

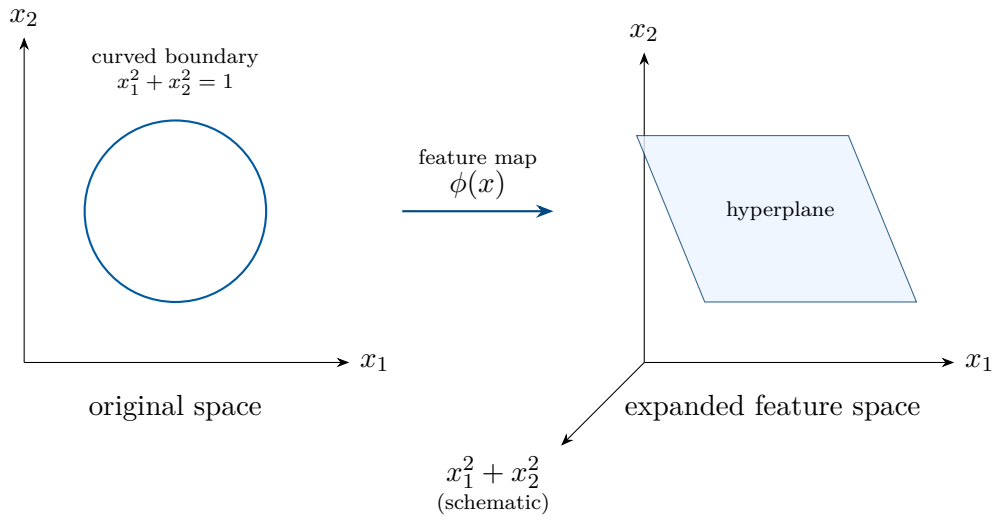

Figure 5: Conceptual comparison between the two geometric views of polynomial logistic regression. In the original input space, the decision boundary may be curved; after the feature mapping, the same classifier corresponds to a flat separating hyperplane in the expanded space.

defined by $\boldsymbol{\theta}^T \mathbf{x} = 0$:

$$-1 + x_1^2 + x_2^2 = 0 \quad \Longleftrightarrow \quad x_1^2 + x_2^2 = 1.$$

Note

The equation $x_1^2 + x_2^2 = 1$ represents all points at distance 1 from the origin; this is precisely the definition of a unit circle. The boundary is a circle not because we designed it to be, but because the data and the squared features together led the algorithm to that solution.

Since $\theta_3 = 1 > 0$ and $\theta_4 = 1 > 0$, the linear combination $-1 + x_1^2 + x_2^2$ is positive outside the unit circle and negative inside. Therefore, $\sigma(-1 + x_1^2 + x_2^2) \geq 0.5$ (predicting $\hat{y} = 1$) for points *outside* the circle, and $\sigma(-1+x_1^2+x_2^2) < 0.5$ (predicting $\hat{y} = 0$) for points *inside* the circle. Reversing the signs of θ_3 and θ_4 would flip the classification directions.

Figure 6 illustrates how a linear model in an expanded feature space can induce a nonlinear decision boundary in the original input space. Although logistic regression is intrinsically a linear classifier, the inclusion of polynomial features (x_1^2 and x_2^2) allows the model to represent a circular boundary. In this example, the learned decision boundary corresponds to the unit circle, separating the two classes: points inside the circle are classified as $\hat{y} = 0$ (orange squares), while points outside are classified as $\hat{y} = 1$ (blue dots). This figure highlights how feature engineering can increase model expressiveness, enabling linear models to capture nonlinear patterns without changing the underlying learning algorithm.

4.3 More Complex Boundaries

By including higher-degree polynomial features and interaction terms, the decision boundary can take on virtually arbitrary shapes. For instance, with features

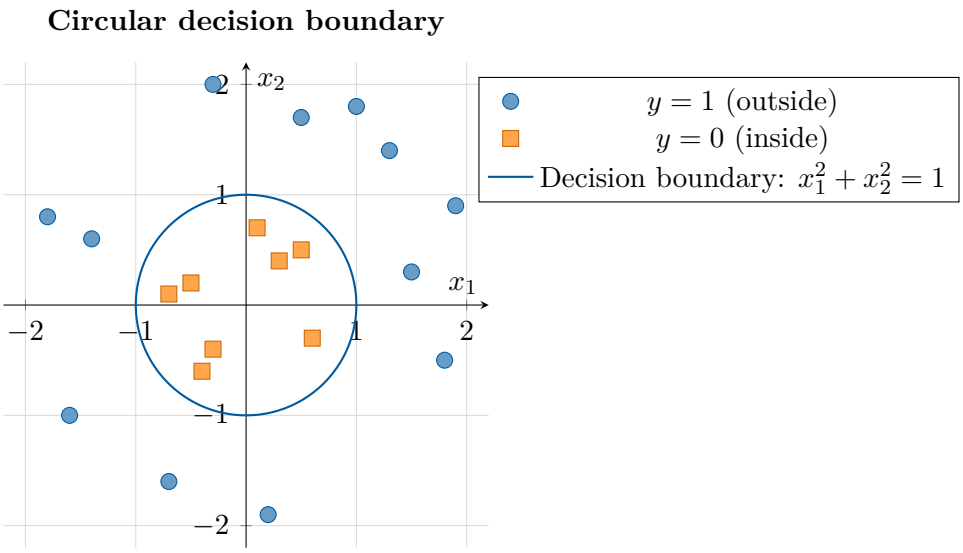

Figure 6: A circular decision boundary arising from logistic regression with polynomial features x_1^2 and x_2^2. With $\boldsymbol{\theta}^T = (-1, 0, 0, 1, 1)$, the boundary is the unit circle. The model predicts $\hat{y} = 1$ (positive class, blue dots) for points outside the circle, and $\hat{y} = 0$ (negative class, orange squares) for points inside.

$$x_1,\ x_2,\ x_1x_2,\ x_1^2x_2,\ x_1^2x_2^2,\ x_1^3x_2^2,\ \ldots,$$

the boundary $\boldsymbol{\theta}^T\boldsymbol{\phi}(x) = 0$ can approximate ellipses, hyperbolas, and much more irregular curves, depending on the learned parameters.

Non-Linear Classification via Polynomial Features

By expanding the feature vector to include polynomial terms and interaction features, logistic regression can learn **non-linear decision boundaries** in the original feature space. The algorithm itself remains unchanged; only the input representation is enriched. The decision boundary in the original space corresponds to a hyperplane in the expanded feature space.

Entry Ticket 3

Refer to non-linear decision boundaries via polynomial features.

1. Given $\theta_0 = -4$, $\theta_3 = 1$, $\theta_4 = 1$ (all other parameters zero) for the hypothesis $h_\theta(x) = \sigma(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$, write the equation of the decision boundary, identify its geometric shape, and compare it to the unit-circle example in the notes. Sketch the new boundary.
2. A classmate claims: "In 3D, with features x_1, x_2, x_3 and the added features x_1^2, x_2^2, x_3^2, a learned boundary could be a sphere." Is this claim correct? If so, what parameter values would produce it?
3. **(Predict the behaviour)** Suppose we progressively increase the degree d in a logistic regression model on a fixed training set. Describe qualitatively how the decision boundary in the original feature space is expected to change. At what point might this become a problem?

5 Overfitting and the Bias–Variance Trade-Off

5.1 The Two Failure Modes

Enriching the feature space with polynomial terms is powerful, but it must be done carefully. There are two opposite failure modes:

- **Underfitting (high bias)**: The model is too simple to capture the true structure in the data. A degree-1 polynomial fitted to inherently quadratic data is a classic example. The model has systematically large errors on both the training set and any new data; no amount of additional training will help.
- **Overfitting (high variance)**: The model is so complex that it memorises the training data, including its noise, instead of learning the underlying pattern. A very high-degree polynomial fitted to a small dataset typically overfits: it passes through (or near) every training point but performs poorly on new, unseen data.

Why Overfitting Happens

With many polynomial features, the model has enough flexibility to pass through every training point, including the ones that are slightly off due to measurement noise or random variation.
The model is fitting noise, not signal.
Since noise does not generalise, the model's predictions on new data are poor despite its excellent performance on the training set.

Interactive visualisation

A standalone interactive simulator (HTML/JavaScript) accompanies these notes. It lets you choose the degree d of a polynomial and then fits it to a fixed dataset.

5.2 The Bias–Variance Trade-Off

The concepts of underfitting and overfitting are captured formally by the **bias–variance trade-off**:

- **Bias** measures how much the model's predictions are systematically wrong because the model class is too restricted to represent the truth. High bias $\Rightarrow$ underfitting.
- **Variance** measures how sensitive the model is to fluctuations in the specific training set used. High variance $\Rightarrow$ overfitting.

As the polynomial degree d increases, bias decreases (the model becomes more expressive) but variance increases (the model becomes more sensitive to the particular training set used). The goal is to find the degree d that minimises the *total* prediction error on unseen data, the sweet spot between underfitting and overfitting.

5.3 Model Selection via Validation Error

In practice, d is a hyperparameter: it is not learned from the training data by gradient descent. Instead, we select it by training models with different values of d and evaluating

their performance on a *held-out validation set* that was not used during training. This provides an unbiased estimate of how well each model generalises.

Figure 7 illustrates the standard model-selection pattern for polynomial degree. As the model becomes more expressive, training error tends to fall steadily. Validation error, however, typically decreases only up to a certain point and then rises once the model begins to fit noise. The degree selected in practice is therefore the one that minimises validation error, not training error.

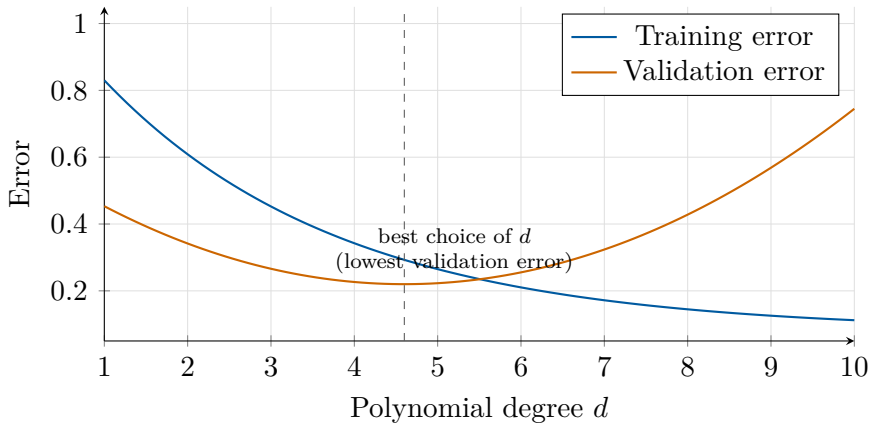

Figure 7: Typical behaviour of training and validation errors as polynomial degree increases. Training error usually decreases monotonically as the model becomes more expressive, whereas validation error often first decreases and then increases once overfitting begins. Model selection chooses the degree that minimises validation error.

The chosen degree is the one that achieves the lowest validation error. Importantly, the test set (i.e., the final held-out data used to report performance) must never be used to make model selection decisions; otherwise it would no longer be a fair measure of generalisation.

Note

The same validation framework applies to any hyperparameter introduced in these notes, including the regularisation parameter λ discussed in the next section. Models learned with different values of λ (or d) are trained on the same training set and compared by validation error; the best model is then re-evaluated once on the test set.

Entry Ticket 4

Refer to the bias–variance trade-off.

1. A model achieves very low training error but high validation error. Is it underfitting or overfitting? List two concrete actions you could take to address this.
2. A model achieves high training error and high validation error. Is it underfitting or overfitting? List two concrete actions you could take to address this.
3. **(Predict the behaviour)** As the polynomial degree d increases from 1 to a very large value, describe qualitatively how the training error and the valida-

tion error are each expected to change. Sketch the two curves on a single plot with degree on the horizontal axis.

4. Why must the test set never be used for model selection? What problem arises if it is?

6 Regularisation

6.1 Motivation

One principled way to control overfitting without manually selecting the polynomial degree is **regularisation**: we add a penalty term to the cost function that discourages large parameter values.

Geometric Intuition

Large parameter values create steep, wiggly fitted curves or highly convoluted decision boundaries. Regularisation penalises large parameter values, which forces the model to produce *smoother* functions, those that are less likely to overfit noise and more likely to generalise.

When polynomial features are used, the parameters associated with high-degree terms often grow disproportionately large to fit noisy training points. Regularisation applies the greatest suppression to these terms because they are the ones most prone to large coefficients, effectively keeping the model complexity in check without explicitly removing high-degree features.

Figure 8 illustrates the qualitative effect of regularisation on a polynomial model. When λ is small, the fitted function has more freedom to bend sharply in order to follow the training data closely. As λ increases, the coefficients are shrunk toward zero, and the resulting fit becomes smoother and less prone to capturing noise.

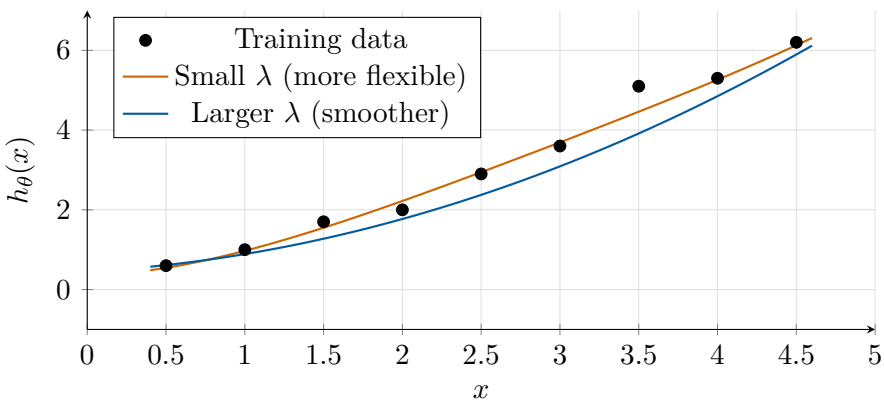

Figure 8: Schematic effect of L2 regularisation on a polynomial model. With weak regularisation, coefficients are freer to become large, which can produce a more wiggly fitted function. Stronger regularisation shrinks the coefficients toward zero and tends to produce smoother, less variable fits.

6.2 L2 Regularisation (Ridge)

The most common approach is **L2 regularisation** (also known as *Ridge*), which adds the sum of squared parameter values (excluding the bias θ_0) to the cost:

L2 Regularised Cost (Regression)

$$J(\boldsymbol{\theta}) = \underbrace{\frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta(\phi^{(i)}) - y^{(i)}\right)^2}_{\text{data fit}} + \underbrace{\frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2}_{\text{regularisation penalty}} \tag{2}$$

where $\lambda \geq 0$ is the **regularisation parameter** (a hyperparameter).

The two terms play opposing roles: the first term encourages the model to fit the training data well; the second term penalises complexity by pushing parameter magnitudes toward zero. The hyperparameter λ controls the balance:

- $\lambda = 0$: no regularisation; the model fits the training data without any constraint on parameter magnitudes. Risk of overfitting.
- $\lambda \to \infty$: the penalty dominates; all parameters $\theta_1, \ldots, \theta_n$ are pushed toward zero, and the hypothesis approaches the trivial constant model $h_\theta(x) \approx \theta_0$. Risk of underfitting.
- Intermediate λ: a balance between fitting the data and keeping parameters small (the goal).

Note

The regularisation term penalises $\theta_1, \ldots, \theta_n$ but **not** θ_0 (the intercept). This is standard practice: penalising the bias would shift all predictions indiscriminately without reducing model complexity.

6.3 Regularised Gradient Descent

Adding the L2 penalty to the regression cost changes the gradient for $j \geq 1$:

Regularised Gradient Descent for Regression

Repeat until convergence:

$$\theta_0 \leftarrow \theta_0 - \frac{\alpha}{m}\sum_{i=1}^{m}\left(h_\theta(\phi^{(i)}) - y^{(i)}\right) \tag{3}$$

$$\theta_j \leftarrow \theta_j\left(1 - \frac{\alpha\lambda}{m}\right) - \frac{\alpha}{m}\sum_{i=1}^{m}\left(h_\theta(\phi^{(i)}) - y^{(i)}\right)\phi_j^{(i)} \qquad (j \geq 1) \tag{4}$$

where $\phi_j^{(i)}$ denotes the j-th expanded feature value for example i.

The factor $\left(1 - \frac{\alpha\lambda}{m}\right)$ is slightly less than 1, so at each step θ_j is first *shrunk* toward zero before the gradient correction is applied. This shrinkage is the regularisation effect in action.

Note the structural parallel with the regularised logistic regression update rules from the previous notes: the expressions are identical in form. Regularisation works the same way whether the downstream model is linear or logistic regression: it always adds a shrinkage factor to the parameter update.

Entry Ticket 5

Refer to L2 regularisation.

1. What happens to the polynomial parameters $\theta_2, \theta_3, \ldots, \theta_d$ as λ increases? How does this affect the shape of the fitted polynomial?
2. Explain why λ is a hyperparameter rather than a model parameter. How would you choose its value in practice?
3. The factor $(1 - \alpha\lambda/m)$ multiplies θ_j at each update step. If $\alpha = 0.01$, $\lambda = 1$, and $m = 100$, compute the value of this factor. What fraction of θ_j is retained at each step?
4. **(Predict the behaviour)** Suppose you increase d to a very high degree but simultaneously increase λ to a very large value. What do you expect the model to look like? Compare this to a model with low d and $\lambda = 0$.

7 Feature Scaling

7.1 Why Feature Scaling Matters More with Polynomial Features

Feature scaling (transforming input features to a common numerical range) is important in any gradient-based method. With polynomial features, its importance is greatly amplified.

Consider a feature x_i with values in the range $[0,\ 10^3]$. The polynomial expansion introduces:

$$x_i \in [0,\ 10^3], \qquad x_i^2 \in [0,\ 10^6], \qquad x_i^3 \in [0,\ 10^9], \qquad x_i^4 \in [0,\ 10^{12}], \qquad \ldots$$

Higher-degree features can be many orders of magnitude larger than the original ones. This creates several problems:

- **Slow or unstable convergence**: Gradient descent takes very different step sizes along different parameter dimensions, causing oscillations or extremely slow progress.
- **Numerical overflow**: Very large feature values can cause floating-point overflow in computing $\boldsymbol{\theta}^T\boldsymbol{\phi}$ or the cost.
- **Ill-conditioned optimisation**: The cost surface becomes highly elongated, and gradient descent zigzags along the valleys instead of moving directly toward the minimum.

7.2 Practical Guidelines for Scaling with Polynomial Features

The order in which scaling and polynomial expansion are applied matters and depends on the computational pipeline. The most common approaches are:

Practical Guideline

Two orderings are used in practice, both are valid:

1. **Scale first, then expand.** Scale the original features to $[0, 1]$ or zero mean/unit variance, then compute the polynomial features from the scaled values. Since the original features lie in a small range, the higher-degree terms do not grow explosively (e.g., if $x'_i \in [0, 1]$, then $(x'_i)^d \in [0, 1]$ as well). This is the most common approach in practice (e.g., scikit-learn pipelines).
2. **Expand first, then scale all features.** Compute polynomial features from the raw values, then scale the entire expanded feature set. This correctly normalises features that may have very different magnitudes after expansion.

Regardless of the chosen order, the **critical requirement** is:

All transformation parameters (min, max, mean, variance) must be estimated on the training set only.

The same transformation (with the training-set statistics) must then be applied to the validation and test sets. Computing statistics on the test set (or the full dataset) would constitute *data leakage* and yield an overly optimistic estimate of generalisation performance.

Entry Ticket 6

Refer to feature scaling in the context of polynomial features.

1. Suppose $x_1 \in [0, 100]$ and you use min-max normalisation to scale it to $[0, 1]$. What is the range of the normalised $(x'_1)^3$? Compare this to the range of the unscaled x_1^3.
2. Why should the scaling statistics be estimated on the training set only and then applied to the test set using those same values? What problem arises if we recompute them on the test set?
3. Explain intuitively why large differences in feature scales slow down gradient descent. What does the cost surface look like when features are on very different scales?
4. **(Predict the behaviour)** A colleague scales all features *after* computing polynomials up to degree 5 from raw features in the range $[0, 100]$. Another colleague scales the original features first. Without running any code, which pipeline is more likely to encounter numerical difficulties, and why?

8 The ML Pipeline with Polynomial Features

8.1 Where This Fits in the Learning Workflow

Having introduced feature expansion, scaling, and regularisation, it is useful to see how these components integrate with the standard supervised learning pipeline introduced in the linear and logistic regression notes.

Figure 9 makes explicit that polynomial features are a *pre-processing* step. The learning algorithm (gradient descent minimising the (regularised) MSE or log-loss cost) is exactly the same one studied in the previous notes. Nothing changes in the optimisation machinery; only the representation of the data entering that machinery changes.

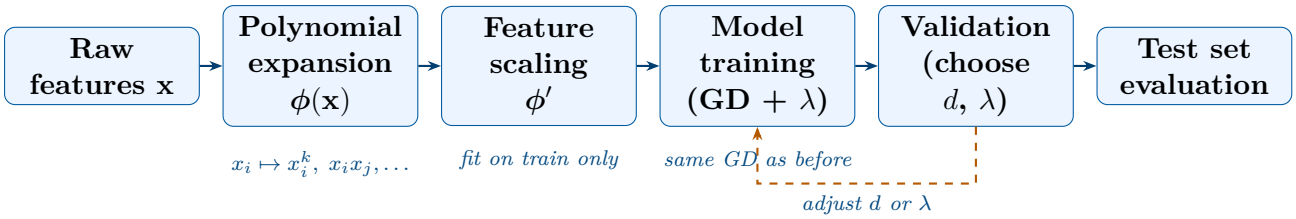

Figure 9: The machine learning pipeline with polynomial features. Feature expansion and scaling are *pre-processing* steps applied before the learning algorithm is run. The learning algorithm itself (gradient descent, cost function, regularisation) remains unchanged from the linear and logistic regression notes. Hyperparameters d and λ are selected using validation error, not test error.

9 Feature Engineering Beyond Polynomials

9.1 The Broader Picture

Polynomial features are a special (and relatively simple) case of the general practice of **feature engineering**: using domain knowledge and mathematical creativity to construct input representations that help the learning algorithm discover useful patterns. Feature engineering can take many forms:

- **Polynomial and interaction features**: as described in these notes.
- **Manually crafted domain features**: for example, BMI is defined as mass/height^2; the area of a room is depth $\times$ width. Such features encode expert knowledge directly into the model input.
- **Automatic feature extraction**: algorithms such as Histogram of Oriented Gradients (HoG) and Scale-Invariant Feature Transform (SIFT) automatically extract structured features from raw images without manual specification.
- **Learned features**: deep neural networks learn their own feature representations end-to-end from the raw input data, largely automating the feature engineering process.

9.2 A Forward Look: Kernel Methods

Polynomial features can be seen as a conceptual bridge to more advanced techniques. In **kernel methods** (such as Support Vector Machines with polynomial or RBF kernels), the idea of mapping inputs to a richer feature space is taken to its logical extreme: the model implicitly operates in a feature space that can be infinite-dimensional, without ever constructing the expanded features explicitly. This is made computationally feasible by the *kernel trick*, which will be explored in later notes.

9.3 Limitations and When to Move On

Polynomial feature expansion has practical limitations:

- The number of features grows combinatorially with n and d, making training slow and memory-intensive for large n or high d.

- The risk of overfitting increases with the number of features; regularisation can mitigate but not eliminate this risk.
- Some non-linear patterns (e.g., complex image recognition tasks) cannot be captured efficiently by polynomial features, no matter how high the degree.

In such cases, more powerful algorithms (support vector machines, random forests, gradient boosting, or deep neural networks) are preferable.

10 Summary

The table below summarises the key concepts introduced in these notes.

Concept	**Description**
Polynomial hypothesis	$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_d x^d$
Expanded feature vector	$\boldsymbol{\phi}(x) = (1,\ x_1,\ x_2,\ x_1^2,\ x_1x_2,\ x_2^2,\ \ldots)$
Linear in parameters	h is linear in θ; features may be non-linear in x
Decision boundary	$\boldsymbol{\theta}^T\boldsymbol{\phi}(x) = 0$ (non-linear in original x; hyperplane in ϕ-space)
Underfitting (high bias)	Model too simple; large error on training and test data
Overfitting (high variance)	Model too complex; low training error, high test error
L2 regularisation	Add $\frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$ to the cost
Regularisation parameter	$\lambda \geq 0$: larger $\lambda \Rightarrow$ simpler model
Feature scaling	Essential; apply consistently; fit statistics on training set only

Key Takeaways

1. Linear and logistic regression in their vanilla forms are limited to **linear decision boundaries**, which are insufficient for many real-world datasets.
2. By constructing **polynomial and interaction features** from the original inputs, we enrich the feature space so that the same gradient descent algorithms can learn non-linear relationships in the original space.
3. Polynomial regression is a special case of linear regression: the hypothesis is **linear in the parameters $\boldsymbol{\theta}$** applied to the expanded feature vector. The learning algorithm does not change, only the input representation does.
4. Adding polynomial features increases model expressiveness but also increases the risk of **overfitting**. The polynomial degree d is a hyperparameter selected using validation error.
5. **L2 regularisation** penalises large parameter values, preventing overfitting without manually reducing the polynomial degree. It disproportionately suppresses high-degree terms, which are most prone to large coefficients.
6. **Feature scaling** is critically important when polynomial features are used, because high-degree terms can have vastly different magnitudes. Scaling

statistics must always be estimated on the training set only.

7. Feature engineering is a broad practice. Polynomial features are a conceptual bridge to more advanced techniques such as kernel methods, where the feature space can be made implicitly infinite-dimensional.